

From Personas & Requirements to Use Cases & UML

PlayGCC – Unified Multi-Sport Booking & Matchmaking Platform

CS3151 Software Engineering – UML Lab

Done by: Hamed Almehdhar, Hasan Albaiti, Moayad Omar

Prepared for course submission – Fall 2026

Context

This lab builds directly on PlayGCC's existing Business Model Canvas, market study, personas (Sara the Social Player, Faisal the Recurring Group Organizer, Munira the Venue Manager), and the 9 user stories and 18 SMART requirements developed in the previous requirements-engineering lab. Those artifacts are transformed here into actors, a use case model, and a consistent set of UML diagrams (activity, sequence, class/domain model), plus two bonus diagrams (state machine and an alternate-flow system sequence diagram).

Tooling note: diagrams were produced with PlantUML (rendered locally), one of the tools explicitly listed as acceptable for this lab. Full PlantUML source for every diagram is included in the Appendix for reproducibility.

D1. Actor Definitions & Use Case List

Actor Cards

Actor	Description	Triggers / Interests	Challenges	Notes
Player	Authenticated end user who books venues and looks for people to play with	Wants to book quickly, find a trustworthy player, avoid fronting the full cost	Short a player at the last minute; cost-splitting friction; trusting a stranger	Personas: Sara, Faisal
Corporate Coordinator	HR/office admin booking recurring slots for employee wellness (specialises Player)	Wants to manage company wellness bookings in bulk, in one place	Administrative overhead of coordinating many employees' games	New B2B segment identified in the feasibility study
Venue Manager	Staff who manage a venue's court availability, pricing, and bookings	Wants to maximise utilisation and cut manual booking admin	Empty off-peak slots; no visibility into demand; manual phone/spreadsheet bookings	Persona: Munira
Payment Gateway «system»	External processor for Mada / STC Pay / Apple Pay	Needs a valid, well-formed charge request	Declined transactions, fraud checks, latency	External system actor
Identity Verification Service «system»	Nafath — national digital identity platform	Needs a valid verification request tied to a national ID	Service downtime; mismatched data	External system actor
Notification Scheduler «timer»	Internal scheduled process that checks recurring-booking renewals and triggers reminders	Runs on a fixed schedule (e.g. nightly)	N/A — internal timer actor	Drives UC-9

Use Case List (prioritised, MoSCoW)

ID	Use Case Name	Primary Actor	Goal / Outcome	Priority
UC-1	Book Venue Slot	Player	A venue court/slot is reserved and paid for	M – Must
UC-2	Request Quick Match	Player	Player is matched with a verified player and the slot is booked	M – Must
UC-3	Manage Recurring Booking	Player	A weekly slot is automatically re-booked for the group	S – Should
UC-4	Rate Player After Match	Player	Both players' ratings are recorded and profiles updated	S – Should
UC-5	Manage Venue Availability & Pricing	Venue Manager	Court schedule and off-peak pricing are updated	M – Must
UC-6	View Demand Forecast	Venue Manager	Forecast is viewed to inform pricing decisions	S – Should
UC-7	Manage Staff Accounts	Venue Manager	Staff accounts are created with correct dashboard permissions	C – Could
UC-8	Book Corporate Wellness Package	Corporate Coordinator	Recurring company bookings are scheduled in bulk	C – Could
UC-9	Receive Slot-Unavailable Notification	Player	Player is alerted in time to rebook before losing the group's slot	S – Should

Won't (for now): city leaderboards / community challenges — a retention feature explicitly deferred past MVP in the market study.

Traceability Matrix (Use Case → User Story → Persona)

UC ID	Use Case	Originating User Story / Source	Persona
UC-1	Book Venue Slot	US-4 (split payment) – foundational to every booking flow	Faisal, Sara, Munira
UC-2	Request Quick Match	US-1	Sara
UC-3	Manage Recurring Booking	US-3, US-9	Faisal
UC-4	Rate Player After Match	US-7 (skill ratings) – trust & safety requirement	Sara
UC-5	Manage Venue Availability & Pricing	US-5, US-8	Munira
UC-6	View Demand Forecast	US-5	Munira
UC-7	Manage Staff Accounts	US-6	Munira
UC-8	Book Corporate Wellness Package	Corporate wellness revenue stream (Feasibility Study, Section 3)	Corporate Coordinator
UC-9	Receive Slot-Unavailable Notification	US-9	Faisal

D2. Use Case Specifications

A. Brief Specifications (all use cases)

Use Case	Primary Actor	Pre-conditions	Post-conditions	Main Success Scenario
UC-1 Book Venue Slot	Player	Player is logged in; slot is available	Slot reserved; payment recorded; confirmation sent	Player selects a slot, confirms, pays, and receives confirmation.
UC-2 Request Quick Match	Player	Player is verified; sport & time window specified	Player matched and slot booked, or added to waitlist	System finds a verified candidate, both accept, slot is booked and paid.
UC-3 Manage Recurring Booking	Player	Player has an active recurring booking rule	Slot re-booked weekly, or player notified if unavailable	System automatically re-books the same slot each week until the end date.
UC-4 Rate Player After Match	Player	A match/booking has been completed	Rating stored; counterpart's profile rating updated	Both players submit a rating and short comment after the match.
UC-5 Manage Venue Availability & Pricing	Venue Manager	Manager is authenticated on the venue dashboard	Court schedule and/or pricing updated	Manager edits available slots and sets off-peak discounts.
UC-6 View Demand Forecast	Venue Manager	Venue has sufficient booking history	Forecast displayed per court and hour	Manager opens the dashboard and views the weekly utilisation forecast.
UC-7 Manage Staff Accounts	Venue Manager	Manager has admin rights on the venue account	Staff account created/updated with a role	Manager adds a staff member and assigns view-only or edit permissions.
UC-8 Book Corporate Wellness Package	Corporate Coordinator	Coordinator's company account is verified	Recurring bulk bookings scheduled for employees	Coordinator selects venues, cadence, and employee group; bookings are created.
UC-9 Receive Slot-Unavailable Notification	Player	A recurring booking's renewal check has run	Player notified with alternative slot suggestions	Scheduler detects a renewal conflict and pushes a notification with alternatives.

B. Fully Dressed Use Case 1 (Core / Happy Path)

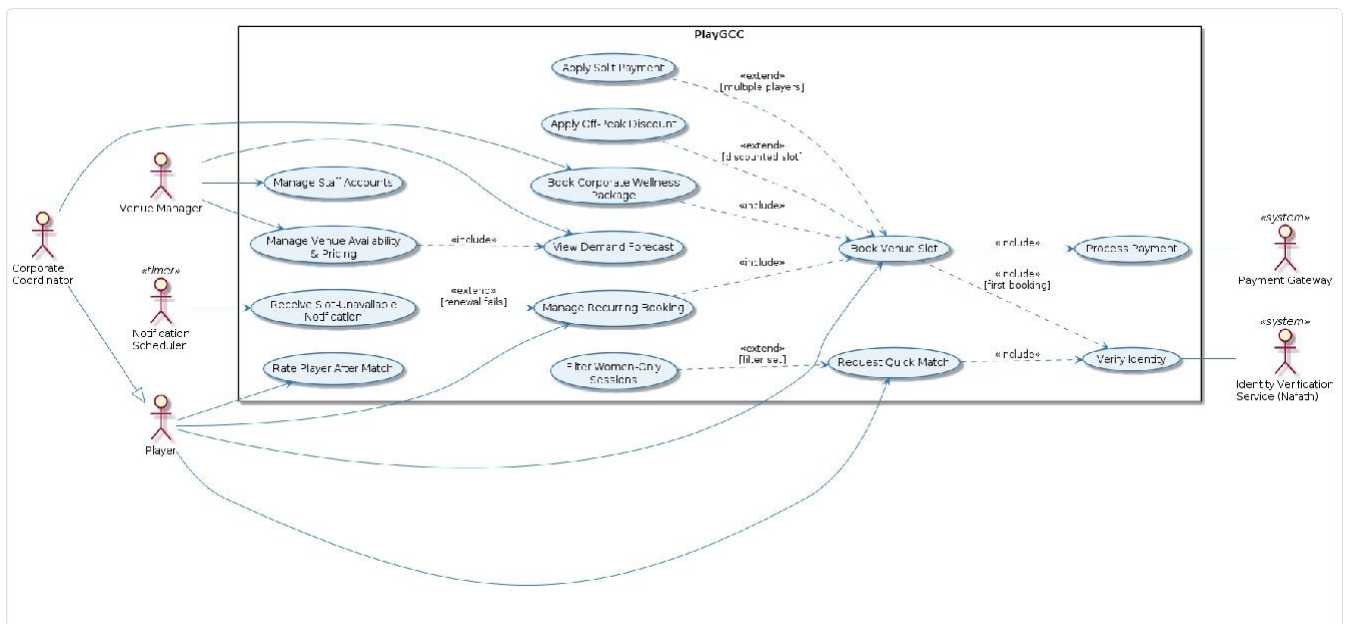
Use Case	UC-1 Book Venue Slot
Activity Name	Book Venue Slot
Primary Actor	Player
Stakeholders & Interests	Player (quick, trustworthy booking); Payment Gateway (valid charge); Venue Manager (accurate utilisation and revenue)
Preconditions (Ip)	Player is authenticated; at least one matching venue slot is available
Postconditions (Op)	Slot is reserved for the player; payment is recorded; booking confirmation is sent
Main Scenario (Flow of Control)	1) Player selects sport, venue, court, and time slot 2) System checks slot availability 3) System calculates price (applying member/off-peak discount if eligible) 4) Player confirms and pays 5) System reserves the slot and confirms the booking
Extensions (Alternatives)	3a) Slot no longer available → 3a1) System suggests nearest alternative slots 4a) Payment fails → 4a1) System offers retry or an alternate payment method (see D9, alternate-flow sequence diagram) 1a) Player has not completed identity verification → 1a1) System prompts Nafath verification before booking (first booking only)
Special Requirements	Payment must complete within 2 seconds of confirmation (per REQ-style NFR); booking confirmation delivered via push notification and WhatsApp
Constraints	Slot must not be double-booked (VenueAvailability enforces a reservation lock) Member discount (if applicable) applied automatically before payment Split payment (multi-player) is an extension, not part of the base flow
Open Issues	How long an unpaid slot hold should be kept before auto-releasing back to availability

C. Fully Dressed Use Case 2 (Complex / Edge)

Use Case	UC-2 Request Quick Match
Activity Name	Request Quick Match
Primary Actor	Player
Stakeholders & Interests	Requesting Player (fast, safe match); Candidate Player (relevant, verified invite); Identity Verification Service (valid verification record)

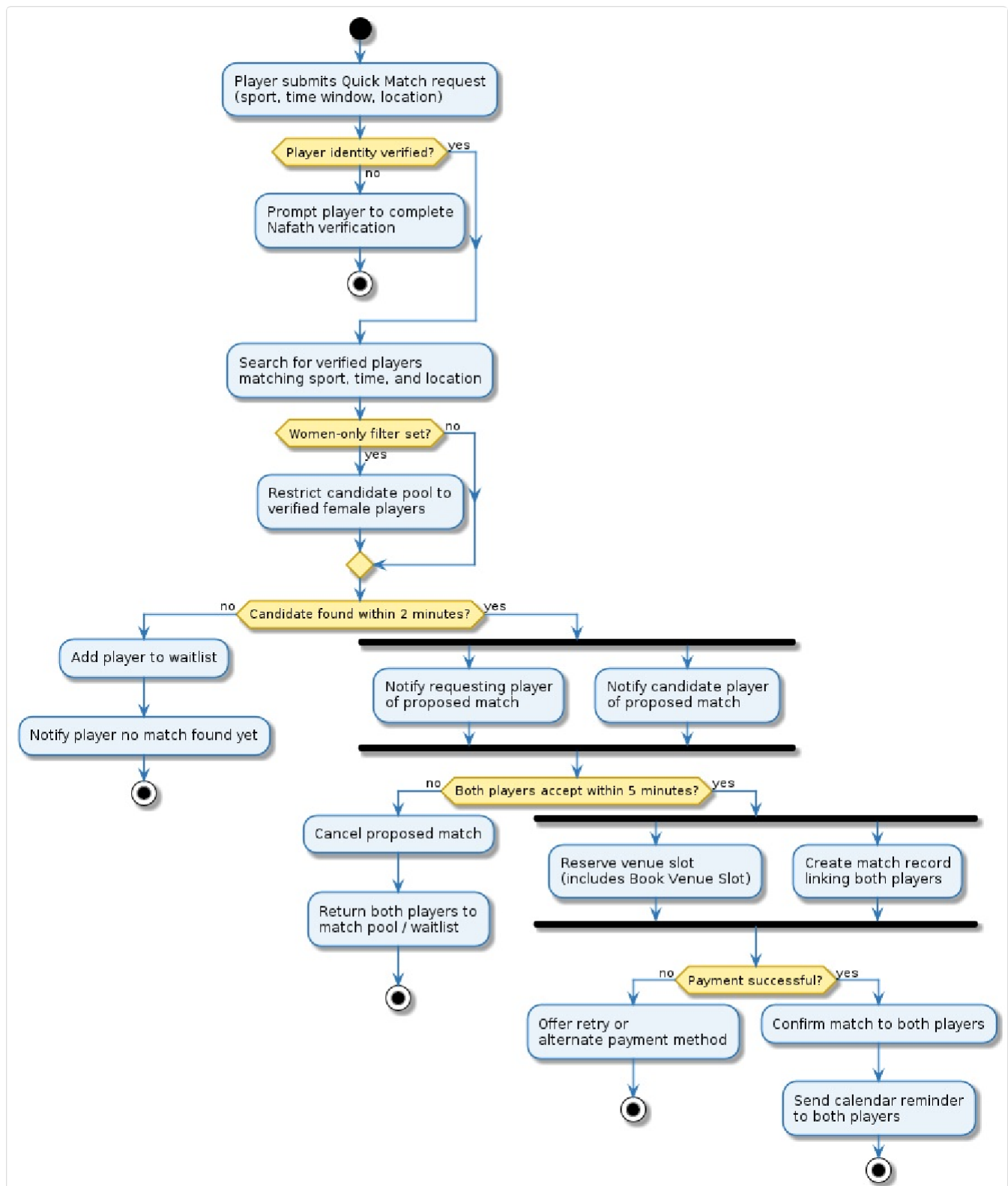
Preconditions (Ip)	Player is authenticated and has completed identity verification
Postconditions (Op)	Either a confirmed, paid booking links both players, or the player is placed on a waitlist with a notification
Main Scenario (Flow of Control)	1) Player submits a Quick Match request (sport, time window, location, optional women-only filter) 2) System searches for verified players matching the criteria 3) System proposes the match to both players in parallel 4) Both players accept within the response window 5) System reserves the venue slot and creates the match record (includes Book Venue Slot) 6) System processes payment and confirms the match to both players
Extensions (Alternatives)	2a) No candidate found within 2 minutes → 2a1) Player is added to a waitlist and notified 4a) One or both players decline, or do not respond within 5 minutes → 4a1) Proposed match is cancelled; both players are returned to the match pool 1a) Women-only filter is set → 1a1) Candidate pool is restricted to verified female players (extend) 6a) Payment fails → 6a1) System offers retry or alternate payment method
Special Requirements	Match search must return a candidate (or exhaust the search) within 2 minutes; only identity-verified players may be suggested as candidates
Constraints	A player can only be proposed one active match at a time Skill rating must be visible to both players before they accept Waitlisted requests expire after 24 hours if unmatched
Open Issues	How to handle repeated no-shows during the accept window without penalising a player who simply had poor connectivity

D3. Use Case Diagram



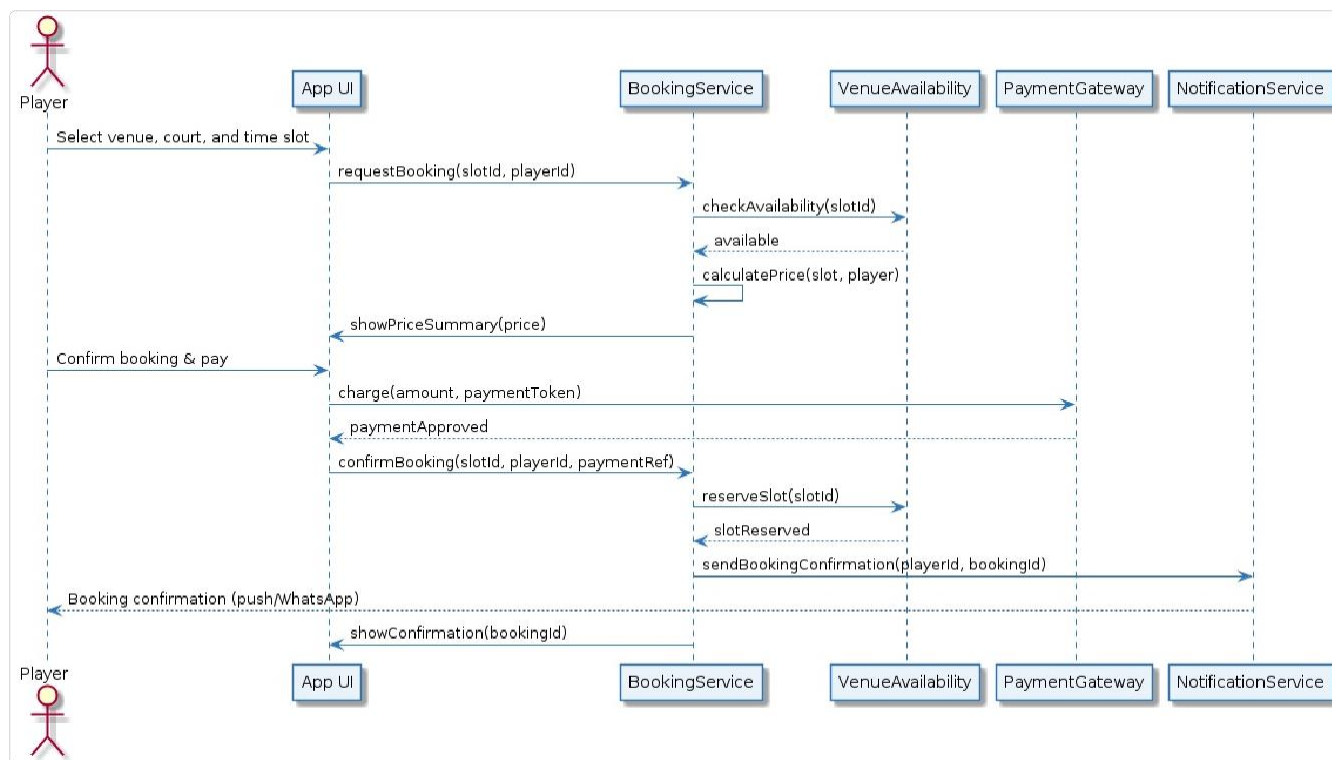
System boundary: PlayGCC. Shows all 9 use cases, include/extend relationships, the Corporate Coordinator generalisation from Player, and the three external/system actors.

D4. Activity Diagram — Request Quick Match (most complex use case)



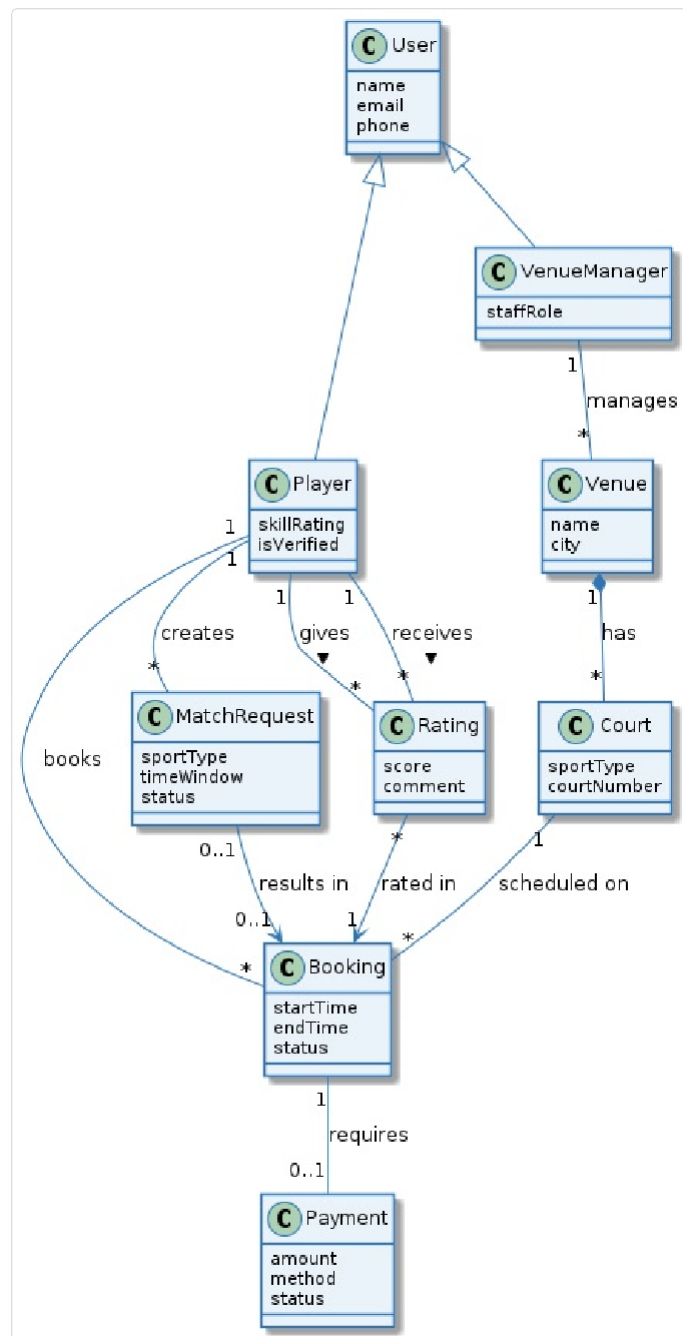
Models the Quick Match workflow: verification gate, candidate search with a timing guard, the women-only filter branch, a fork/join to notify both players in parallel, the accept/decline branch, and a second fork/join to reserve the slot and create the match record before payment.

D5. Sequence Diagram — Book Venue Slot (main success scenario)



Object interaction for the happy path of UC-1, matching the 5-step Main Success Scenario in the fully dressed specification: select & pay; check availability, price, pay, reserve, confirm.

D6. Domain Model (Class Diagram)



Domain concepts extracted from the requirements and use cases, not the database schema. User generalises into Player and VenueManager (mirroring the accepted Customer/Member pattern from the Movie Shop exercise). Composition: Venue owns Court. Multiplicities shown on every association.

D7. Consistency Checklist & Peer Review

Consistency Checklist

Check	Status	Evidence
Every use case is traceable to at least one actor and appears in the Use Case Diagram	✓ Pass	All 9 use cases in the D1 list appear in D3, each connected to Player, Venue Manager, or Corporate Coordinator.
The Sequence Diagram messages align with the Main Success Scenario steps	✓ Pass	D5's 5 message groups (select → check → price → pay → reserve/confirm) map 1:1 to UC-1's 5-step Main Scenario in D2.
The Activity Diagram branches match Extensions/guards in the use case	✓ Pass	D4's "no candidate found," "player declines," and "payment fails" branches correspond directly to UC-2's 2a, 4a, and 6a extensions in D2.
All domain classes appear in at least one use case/sequence; names are consistent	✓ Pass	Player, Booking, Payment, and Venue/Court all appear by the same name across D5 (sequence) and D6 (class diagram); MatchRequest and Rating map to UC-2 and UC-4 respectively.
Non-functional constraints in the use case appear as notes/stereotypes where relevant	✓ Pass	UC-1's "payment < 2s" and UC-2's "match search < 2 min" special requirements are reflected in the activity diagram's timing guards.

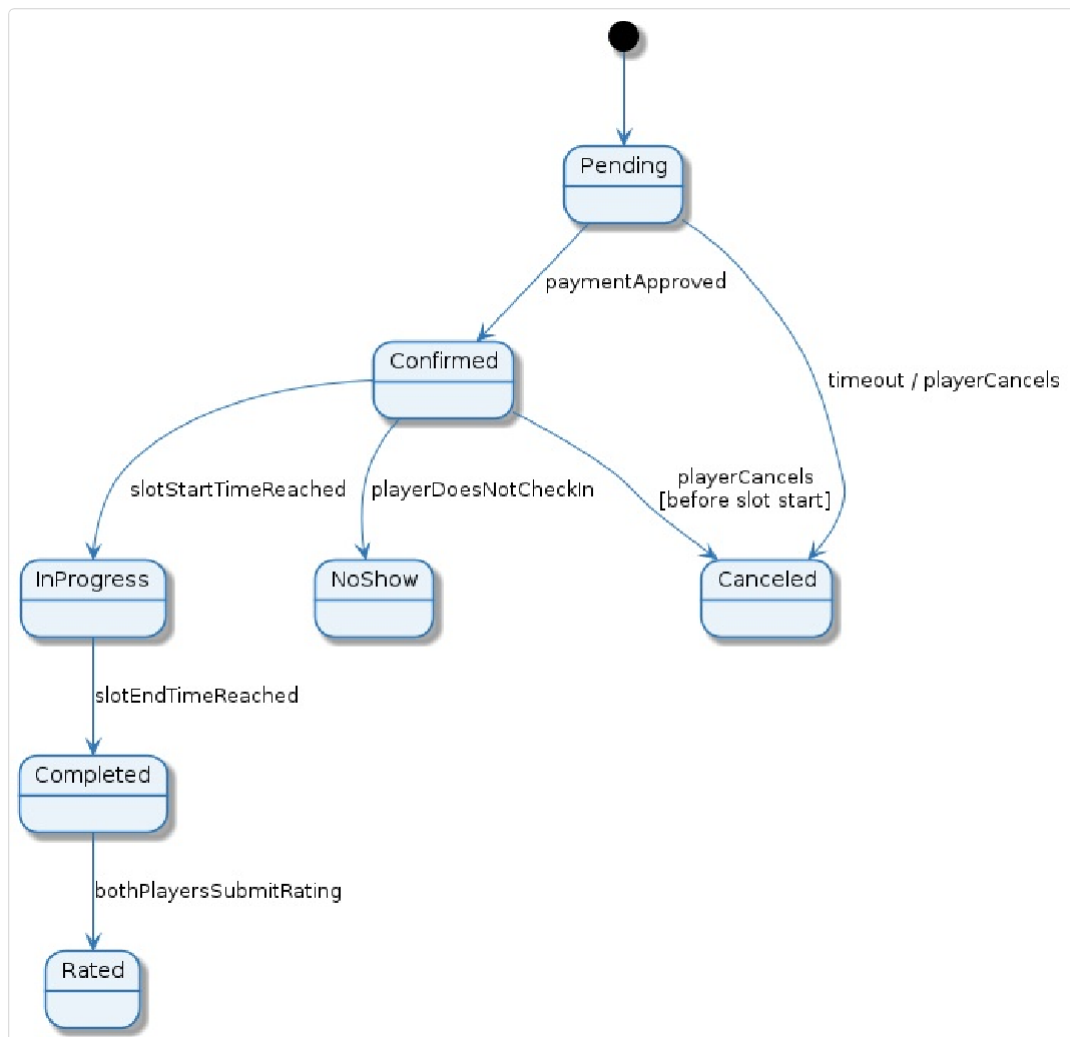
Peer-Review Checklist

This lab was completed individually rather than in a team, so the checklist below is a self-review rather than an independent peer review. Before final submission, it is recommended this be reviewed by an actual classmate or teammate, per the lab's intended process.

Item	Self-check
Actors cover all personas and external systems	☐
Use case names are goal-level and unambiguous	☐
Includes/extends used sparingly and correctly	☐
Activity/Sequence diagrams match narratives	☐
Class diagram avoids UI/DB details; multiplicities make sense	☐
Clear pre/post-conditions and success guarantees	☐
Consistency across all artifacts	☐
Clarity, readability, and professional style	☐

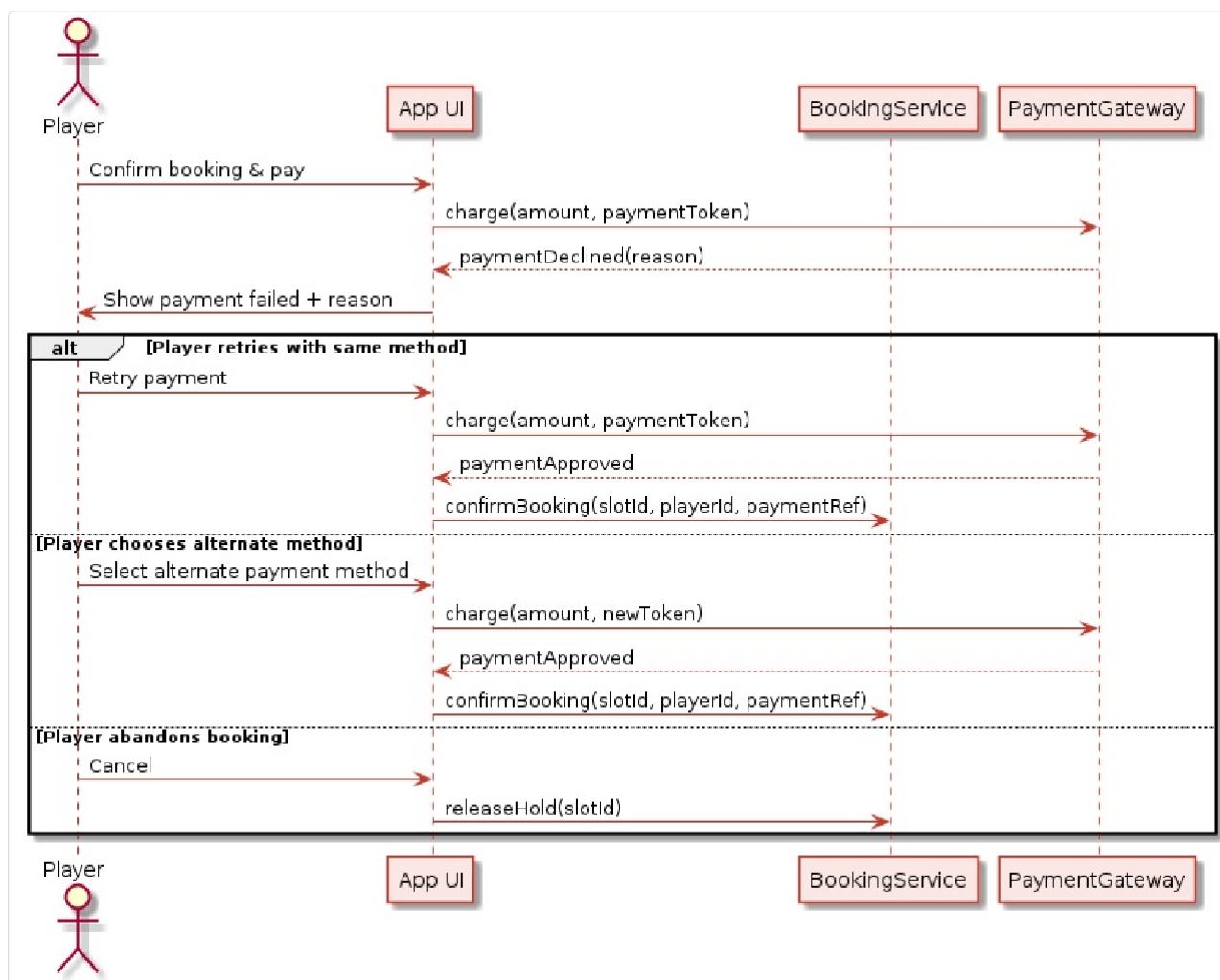
Extension (Optional, +5 pts each)

State Machine Diagram — Booking Lifecycle



Lifecycle of a Booking from creation through payment, play, and rating, including cancellation and no-show paths.

System Sequence Diagram — Alternate Flow (Failed Payment)



Covers UC-1 extension 4a: payment decline, followed by the three alternatives available to the player (retry, alternate method, or abandon booking).

Appendix — PlantUML Source

Included for reproducibility, as invited by the lab instructions.

D3. Use Case Diagram

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome
skinparam backgroundColor white
skinparam usecase {
    BackgroundColor #EAF3FA
    BorderColor #2E75B6
    ArrowColor #2E75B6
}

actor Player
actor "Corporate\nCoordinator" as Coord
actor "Venue Manager" as VM
actor "Payment Gateway" as PG <<system>>
actor "Identity Verification\nService (Nafath)" as IDV <<system>>
actor "Notification\nScheduler" as Timer <<timer>>

Coord --> Player

rectangle "PlayGCC" {
    usecase "Book Venue Slot" as UC1
    usecase "Request Quick Match" as UC2
    usecase "Manage Recurring Booking" as UC3
    usecase "Rate Player After Match" as UC4
    usecase "Manage Venue Availability\n& Pricing" as UC5
    usecase "View Demand Forecast" as UC6
    usecase "Manage Staff Accounts" as UC7
    usecase "Book Corporate Wellness\nPackage" as UC8
    usecase "Receive Slot-Unavailable\nNotification" as UC9
    usecase "Process Payment" as UCa
    usecase "Verify Identity" as UCb
    usecase "Apply Split Payment" as UCc
    usecase "Apply Off-Peak Discount" as UCd
    usecase "Filter Women-Only\nSessions" as UCe
}

Player --> UC1
Player --> UC2
Player --> UC3
Player --> UC4
Coord --> UC8
VM --> UC5
VM --> UC6
VM --> UC7

UC1 ..> UCa : <<include>>
UC1 ..> UCb : <<include>>\n[first booking]
UCc ..> UC1 : <<extend>>\n[multiple players]
UCd ..> UC1 : <<extend>>\n[discounted slot]
UC2 ..> UCb : <<include>>
UCe ..> UC2 : <<extend>>\n[filter set]
UC3 ..> UC1 : <<include>>
UC9 ..> UC3 : <<extend>>\n[renewal fails]
UC5 ..> UC6 : <<include>>
UC8 ..> UC1 : <<include>>

UCa -- PG
UCb -- IDV
Timer --> UC9

@enduml
```

D4. Activity Diagram

```
@startuml
skinparam backgroundColor white
skinparam activity {
    BackgroundColor #EAF3FA
    BorderColor #2E75B6
    ArrowColor #2E75B6
    DiamondBackgroundColor #FFF2A8
    DiamondBorderColor #C99A00
}

start
:Player submits Quick Match request\n(sport, time window, location);
```

```

if (Player identity verified?) then (no)
  :Prompt player to complete\nNafath verification;
  stop
else (yes)
endif
:Search for verified players\nmatching sport, time, and location;
if (Women-only filter set?) then (yes)
  :Restrict candidate pool to\nverified female players;
else (no)
endif
if (Candidate found within 2 minutes?) then (no)
  :Add player to waitlist;
  :Notify player no match found yet;
  stop
else (yes)
  fork
    :Notify requesting player\nof proposed match;
  fork again
    :Notify candidate player\nof proposed match;
  end fork
  if (Both players accept within 5 minutes?) then (no)
    :Cancel proposed match;
    :Return both players to\nmatch pool / waitlist;
    stop
  else (yes)
    fork
      :Reserve venue slot\n(includes Book Venue Slot);
    fork again
      :Create match record\nlinking both players;
    end fork
    if (Payment successful?) then (no)
      :Offer retry or\nalternate payment method;
      stop
    else (yes)
      :Confirm match to both players;
      :Send calendar reminder\nto both players;
      stop
    endif
  endif
endif
endif
@enduml

```

D5. Sequence Diagram

```

@startuml
skinparam backgroundColor white
skinparam sequence {
  ArrowColor #2E75B6
  LifeLineBorderColor #2E75B6
  ParticipantBackgroundColor #EAF3FA
  ParticipantBorderColors #2E75B6
}

actor Player
participant "App UI" as UI
participant "BookingService" as BS
participant "VenueAvailability" as VA
participant "PaymentGateway" as PG
participant "NotificationService" as NS

Player -> UI: Select venue, court, and time slot
UI -> BS: requestBooking(slotId, playerId)
BS -> VA: checkAvailability(slotId)
VA --> BS: available
BS -> BS: calculatePrice(slot, player)
BS -> UI: showPriceSummary(price)
Player -> UI: Confirm booking & pay
UI -> PG: charge(amount, paymentToken)
PG --> UI: paymentApproved
UI -> BS: confirmBooking(slotId, playerId, paymentRef)
BS -> VA: reserveSlot(slotId)
VA --> BS: slotReserved
BS -> NS: sendBookingConfirmation(playerId, bookingId)
NS --> Player: Booking confirmation (push/WhatsApp)
BS -> UI: showConfirmation(bookingId)
@enduml

```

D6. Domain Model (Class Diagram)

```

@startuml
skinparam backgroundColor white
skinparam class {
  BackgroundColor #EAF3FA
}

```

```

    BorderColor #2E75B6
    ArrowColor #2E75B6
}

class User {
    name
    email
    phone
}
class Player {
    skillRating
    isVerified
}
class VenueManager {
    staffRole
}
class Venue {
    name
    city
}
class Court {
    sportType
    courtNumber
}
class Booking {
    startTime
    endTime
    status
}
class Payment {
    amount
    method
    status
}
class MatchRequest {
    sportType
    timeWindow
    status
}
class Rating {
    score
    comment
}

User <|-- Player
User <|-- VenueManager

VenueManager "1" -- "*" Venue : manages
Venue "1" *-- "*" Court : has

Player "1" -- "*" Booking : books
Court "1" -- "*" Booking : scheduled on
Booking "1" -- "0..1" Payment : requires

Player "1" -- "*" MatchRequest : creates
MatchRequest "0..1" --> "0..1" Booking : results in

Player "1" -- "*" Rating : gives >
Player "1" -- "*" Rating : receives >
Rating "*" --> "1" Booking : rated in
@enduml

```

Extension. State Machine Diagram

```

@startuml
skinparam backgroundColor white
skinparam state {
    BackgroundColor #EAF3FA
    BorderColor #2E75B6
    ArrowColor #2E75B6
}

[*] --> Pending
Pending --> Confirmed : paymentApproved
Pending --> Canceled : timeout / playerCancels
Confirmed --> InProgress : slotStartTimeReached
InProgress --> Completed : slotEndTimeReached
Confirmed --> Canceled : playerCancels\n[before slot start]
Completed --> Rated : bothPlayersSubmitRating
Confirmed --> NoShow : playerDoesNotCheckIn
@enduml

```

Extension. Alternate-Flow Sequence Diagram

```
@startuml
skinparam backgroundColor white
skinparam sequence {
    ArrowColor #C0392B
    LifeLineBorderColor #C0392B
    ParticipantBackgroundColor #FBE7E4
    ParticipantBorderColor #C0392B
}

actor Player
participant "App UI" as UI
participant "BookingService" as BS
participant "PaymentGateway" as PG

Player -> UI: Confirm booking & pay
UI -> PG: charge(amount, paymentToken)
PG --> UI: paymentDeclined(reason)
UI -> Player: Show payment failed + reason
alt Player retries with same method
    Player -> UI: Retry payment
    UI -> PG: charge(amount, paymentToken)
    PG --> UI: paymentApproved
    UI -> BS: confirmBooking(slotId, playerId, paymentRef)
else Player chooses alternate method
    Player -> UI: Select alternate payment method
    UI -> PG: charge(amount, newToken)
    PG --> UI: paymentApproved
    UI -> BS: confirmBooking(slotId, playerId, paymentRef)
else Player abandons booking
    Player -> UI: Cancel
    UI -> BS: releaseHold(slotId)
end
@enduml
```